

Agentic Programming as Formal Automata: A Chomsky-Hierarchy View of Tool-Using LLM Agents

Scott Weeden

May 23, 2026

Abstract

We propose a unifying vocabulary in which *agentic programs*—policies that interleave observation, reasoning, and tool invocation—are classified by the same structural constraints that separate finite automata, pushdown automata, linear-bounded automata, and Turing machines in the Chomsky hierarchy. Rather than claiming that LLM agents literally implement any one grammar, we define the *trace languages* generated by agent schemata and show how restrictions on control-state cardinality, auxiliary memory, and growth of workspace relative to interaction length yield a four-level taxonomy analogous to Types 3–0. The goal is to make informal claims about “bounded” versus “general” agent stacks, memory, and planning depth formally comparable to classical automata theory, supporting reproducible evaluation design and peer critique.

1 Introduction

Software systems that combine large language models (LLMs) with tools, retrieval, and multi-step control flow are often described as *agentic*: they repeatedly observe an environment, update internal state, and emit actions until a termination condition holds (?). Surveys of “agentic AI” emphasize planning depth, tool diversity, and memory mechanisms, but rarely tie these informal axes to the classical theory of computation.

The Chomsky hierarchy partitions formal languages by generative power and, equivalently, by the automaton class needed to recognize them (?). Type 3 (regular) languages correspond to finite-state control; Type 2 (context-free) to a single stack; Type 1 (context-sensitive) to linear-bounded workspace; Type 0 to unrestricted Turing machines.

This paper does *not* assert that a deployed LLM instantiates a particular grammar on natural language. Instead, we treat an *agentic program* as a specified controller—often a finite template of prompts, parsers, and API calls—whose *interaction traces* form a formal language over finite grammars of observable events (finite sets of observation and action terminals after discretization). Restrictions on that controller’s state space and memory discipline yield a hierarchy of trace languages that parallels Types 3–0. The contribution is definitional: we supply a common formal basis for comparing agent designs, benchmarking, and peer review, in line with separating claims from evidence in multi-model evaluation (?).

Claims and scope

We state comparative claims only at the level of *agent schemata* (Section ??) and their idealized trace languages. Empirical measurements of real LLM agents must map implementations to these schemata; Section ?? outlines how to pre-register such mappings.

2 Preliminaries: Automata, Traces, and Languages

2.1 Observation and action grammars; traces

Fix **finite grammars** for observations and for actions (terminal-only forms realized at runtime). Let Σ_{obs} and Γ_{act} denote the **finite sets of observation terminals** and **atomic action terminals**, respectively (concrete encodings of discretized UI events, API codes, tool names with bounded argument tags). A *step* is a pair (o, a) with $o \in \Sigma_{\text{obs}}$ and $a \in \Gamma_{\text{act}}^*$ (a finite sequence of atomic actions; ε denotes the empty sequence).

A *trace* of length n is a sequence

$$\tau = (o_1, a_1)(o_2, a_2) \cdots (o_n, a_n),$$

where each (o_i, a_i) is a step. The set of all finite traces is $\mathcal{T} = (\Sigma_{\text{obs}} \times \Gamma_{\text{act}}^*)^*$. A *trace language* is any subset $L \subseteq \mathcal{T}$.

2.2 Chomsky hierarchy (recognition view)

For reference: a language $L \subseteq \Sigma^*$ of strings over finite Σ is Type 3 (regular) iff some finite automaton recognizes it; Type 2 iff some nondeterministic pushdown automaton (NPDA) recognizes it; Type 1 iff some linear-bounded automaton (LBA) recognizes it; Type 0 iff some Turing machine recognizes it (?).

2.3 Remark on morphisms

Traces are not raw strings until we fix an injective encoding $\text{enc} : \mathcal{T} \rightarrow \Sigma^*$ into some **string language** over a finite symbol set (classical formal-language machinery). All hierarchy claims below are stated modulo such an encoding that is injective on the trace set under study. In implementation-facing documents, prefer speaking of **grammars**, **terminals**, **states**, **memory**, and **context** rather than informal “alphabet” talk for agent layers.

3 Agent Schemata and Transition Systems

3.1 Configurations

Definition 3.1 (Workspace). *A workspace is a structure $\mathcal{W}$ equipped with a size function $\|\cdot\| : \mathcal{W} \rightarrow \mathbb{N}$ and an update operator that may depend on observations and actions. Examples: (i) a finite register file; (ii) a stack of pending subgoals; (iii) a tape of length $f(n)$ after n steps; (iv) read/write access to an unbounded external store.*

Definition 3.2 (Agent schema). *An agent schema is a tuple*

$$\mathcal{A} = (S, s_0, \mathcal{W}_0, \Sigma_{\text{obs}}, \Gamma_{\text{act}}, \delta, F),$$

where S is a finite set of modes, $s_0 \in S$ is the initial mode, $\mathcal{W}_0$ is the initial workspace, Σ_{obs} and Γ_{act} are the finite observation and action **terminal** sets fixed in Section ?? (each grounded by a finite grammar of allowed surface forms), $F \subseteq S \times \mathcal{W}$ is a set of accepting configurations, and

$$\delta : S \times \mathcal{W} \times \Sigma_{\text{obs}} \rightarrow \mathcal{P}_{\text{fin}}(S \times \mathcal{W} \times \Gamma_{\text{act}}^*)$$

is a (possibly nondeterministic) transition rule. For every (s, w, o) , the set $\delta(s, w, o)$ is finite; its elements are the finite successors (successor mode, workspace, and finite action sequence).

Definition 3.3 (Run and trace language). *A run of $\mathcal{A}$ on observation sequence $o_1, \dots, o_n$ is a sequence of configurations $(s_i, \mathcal{W}_i)$ and actions $a_i \in \Gamma_{\text{act}}^*$ satisfying $(s_{i+1}, \mathcal{W}_{i+1}, a_{i+1}) \in \delta(s_i, \mathcal{W}_i, o_{i+1})$, starting at $(s_0, \mathcal{W}_0)$. The associated trace is $(o_1, a_1) \cdots (o_n, a_n)$. The trace language $L(\mathcal{A})$ is the set of traces of accepting runs (configurations in F) under a fixed termination rule (e.g., halting on a designated stop observation).*

Remark 3.1. *This definition intentionally mirrors automata theory: modes S generalize finite control; $\mathcal{W}$ encodes whatever memory the agent design allows beyond finite control.*

Definition 3.4 (Hyperparameters and configuration). *A configuration is a pair $(s, \mathcal{W}) \in S \times \mathcal{W}$. Hyperparameters are external tunables that fix the schema or its implementation (e.g. decoding temperature, maximum tool-call depth, retrieval top-k, stopping thresholds). Configuration denotes environment-level bindings (paths, API endpoints, credential scopes, profile directories) that instantiate the schema in software without changing the abstract tuple $\mathcal{A}$.*

4 Agent-to-Agent Graphs, Trace Snapshots, and Context Bounds

4.1 Nodes and edges

We distinguish a single agent schema $\mathcal{A}$ (Definition ??) from *deployed instances* that may message one another. Inter-agent channels carry payloads drawn from finite grammars of messages (tool-delegation requests, structured observations, handoff tokens), not from raw unbounded natural language, once a discretization is fixed for analysis.

Definition 4.1 (Agent-to-agent node). *An agent node (or deployment node) is a tuple*

$$\nu = (\eta, s, \mathcal{W}, \kappa),$$

where η is an instance identifier (e.g. URI or UUID), $(s, \mathcal{W}) \in S \times \mathcal{W}$ is the current local configuration of the underlying schema, and κ collects hyperparameters and configuration (Definition ??) instantiated for this node: decoding settings, tool scopes, memory policy, and channel endpoints. Two nodes may share the same abstract schema but differ in η or κ .

Definition 4.2 (Agent-to-agent edge). *Fix a finite grammar of inter-agent messages whose terminal forms populate a finite set $\mathcal{M}$ (message kinds with bounded payloads, after discretization). A directed agent edge is a tuple*

$$e = (\nu_{\text{src}}, \nu_{\text{tgt}}, m, \iota),$$

where $\nu_{\text{src}}, \nu_{\text{tgt}}$ are nodes, $m \in \mathcal{M}$ is the transmitted message, and ι is an interaction index (time step or sequence number) distinguishing parallel edges. An agent-to-agent graph is a pair $G = (V, E)$ with V a finite set of nodes and E a multiset of edges closed under the intended channel topology.

Remark 4.1 (Alignment with “nodes” in software). *In codebases, a “node” often denotes a planner state or graph-search vertex; here, node means a deployed agent instance (Definition ??). A “graph edge” in orchestration layers corresponds to Definition ??; control-flow graphs inside one agent remain internal to δ and $\mathcal{W}$.*

4.2 Trace snapshot languages by schema type

A *state snapshot* records everything the analysis treats as observable about $(s, \mathcal{W})$ at a step (up to the chosen abstraction). Fix a finite set $\mathcal{Z}$ of *snapshot descriptors* (encodings of modes plus bounded summaries of memory shape). A *snapshot sequence* is a word in $\mathcal{Z}^*$.

Definition 4.3 (Type-3 (regular) snapshot traces). *For a regular schema (Definition ??), $\mathcal{W}$ ranges over a finite set. After fixing an injective encoding $\sigma : S \times \mathcal{W} \rightarrow \mathcal{Z}$, the set of feasible snapshot sequences is finite-state: it is the language of a finite automaton over snapshot descriptors (equivalently, a Type-3 language over $\mathcal{Z}$ under the usual string view).*

Definition 4.4 (Type-2 (pushdown-style) snapshot traces). *For a pushdown-style schema (Definition ??), fix a finite set of stack terminals (symbols pushed/popped under δ). A snapshot descriptor includes the finite control mode and a finite encoding of the stack (e.g. as a string over stack terminals). The set of feasible snapshot sequences is stack-structured: it coincides with the behavior of a pushdown system over $\mathcal{Z}$ (analogous to Type-2 generative power on suitable encodings).*

Definition 4.5 (Type-1 (linear-bounded) snapshot traces). *For a linear-bounded schema (Definition ??), fix c, c_0 and an encoding of $\mathcal{W}$ into a tape of length at most $cn + c_0$ after n steps. Snapshot descriptors include the mode and such a bounded tape inscription. Feasible snapshot sequences respect the linear space bound at every prefix; under standard encodings, this matches the spirit of linear-bounded automata (Type 1 in the classical string setting).*

4.3 Context window as finite memory and compaction policies

Real LLM-backed agents store prompts and transcripts in a buffer of bounded length (tokens or other measured units). We model this as a finite workspace component $\mathcal{W}_{\text{ctx}}$ with capacity $M \in \mathbb{N}$, together with a *compaction policy* applied when new content would exceed M .

Let H denote the logical interaction history (observations and actions, or their string rendering) and $B \in \mathcal{W}_{\text{ctx}}$ the stored buffer. A policy is a partial map $\Pi : (B, H_{\text{new}}) \mapsto B'$ such that $\|B'\| \leq M$ whenever defined.

Definition 4.6 (Context-boundary policies). *We distinguish three policies for upper bounds on context:*

1. **Halt at limit:** *if appending H_{new} would exceed M , δ enters a designated halting or error mode and no further tokens are appended.*
2. **Truncate middle (default finite-memory model):** *B' consists of a kept prefix, a kept suffix of the combined old and new material, and an optional fixed marker; middle segments are discarded so $\|B'\| \leq M$. This is the default when describing agents with finite memory realized by LLM context windows.*
3. **Rolling window:** *split capacity between a compacted summary region and a recent verbatim region: $B' = \text{summary}(B, H_{\text{new}}) \parallel \text{recent}(B, H_{\text{new}})$, where `summary` writes into a reserved sub-capacity of $\mathcal{W}_{\text{ctx}}$ (lossy compression of older history), and `recent` preserves the tail up to the remaining budget. Execution continues (unlike halt), but older detail is only represented indirectly in the summary.*

Remark 4.2. *Truncate-middle and rolling-window policies are not unbounded recall: they induce a finite-state or lossy summary of remote history. Claiming Type 0 power therefore requires external stores whose growth is not modeled by a fixed M (Section ??).*

5 A Four-Level Hierarchy of Agent Schemata

We classify agent schemata by constraints on S and $\mathcal{W}$. The analogy to the Chomsky hierarchy is *structural*: each level matches the memory pattern of the corresponding automaton class.

5.1 Type-3 (regular) agent schemata

Definition 5.1 (Regular agent schema). *A schema $\mathcal{A}$ is regular if $\mathcal{W}$ is drawn from a finite set (equivalently $\|\mathcal{W}\|$ is bounded by a constant independent of interaction length) and δ uses only this finite workspace.*

Interpretation: The controller cannot remember an unbounded history except through a finite summary; behavior is Markovian over $(s, \mathcal{W})$. This models fixed prompt templates with bounded counters, finite state machines wrapped around an LLM call, or policies that discard raw history after hashing to a finite codebook.

5.2 Type-2 (pushdown-style) agent schemata

Definition 5.2 (Pushdown agent schema). *A schema is pushdown-style if $\mathcal{W}$ is a stack over a finite set of stack terminals (the push/pop vocabulary fixed by a finite grammar of stack cells), with push/pop determined by δ , and $|S| < \infty$.*

Interpretation: Nested subgoals (e.g., call a tool, then resume parent plan) are first-class. This aligns with hierarchical task networks and nested ReAct-style loops where the depth of the call stack is the salient memory resource (?).

5.3 Type-1 (linear-bounded) agent schemata

Definition 5.3 (Linear-bounded agent schema). *Fix an encoding of past observations into a tape of symbols. A schema is linear-bounded if, after n steps, $\|\mathcal{W}\| \leq cn + c_0$ for constants c, c_0 , and δ never expands the workspace beyond this bound relative to the recorded interaction length n .*

Interpretation: The agent may use scratch space proportional to task size (e.g., storing the full observation transcript over a fixed finite observation vocabulary) but not superlinear growth without external resources.

5.4 Type-0 (Turing-complete) agent schemata

Definition 5.4 (Turing agent schema). *A schema is Turing-complete if $\mathcal{W}$ may include an unbounded read/write store (or an oracle equivalent to unbounded external memory) such that the family of transition relations simulates an arbitrary Turing machine on some encoding of inputs.*

Interpretation: Arbitrary files, growing vector stores, or unconstrained scratch pads make the overall system capable of general computation over finite encodings of observations.

Definition 5.5 (Shortcuts and strategies (physical Type-0 models)). *No physical implementation provides truly unbounded memory. We therefore distinguish the idealized Type-0 schema from deployed agents that approximate unbounded recall using:*

- **Shortcuts:** *morphisms that map long traces or histories to compact handles (chunk identifiers, hashes, pointers into an external store) so that δ can depend on data not carried in the LLM buffer.*
- **Strategies:** *a family of policies over such shortcuts together with retrieval rules (when to read, merge, or refresh external material) that realize effective read/write to an external store within finite per-step time budgets.*

Formally, fix an external store $\mathcal{E}$ with update and query operations. A shortcut is a pair (key, fetch) where key maps a finite summary of history into a finite locator for $\mathcal{E}$, and fetch retrieves a finite fragment into $\mathcal{W}_{\text{ctx}}$ under a chosen context policy (Definition ??). A strategy is a schedule of shortcut applications and δ -steps subject to resource bounds.

Remark 5.1. *Accordingly, empirical claims of “Type-0 behavior” should state which external $\mathcal{E}$ and which shortcut/strategy layer are assumed; the idealized hierarchy in Theorem ?? is about schemata, while deployed systems are always finite at any instant.*

Theorem 5.1 (Strict inclusion (idealized)). *Under standard encodings and assuming non-trivial terminal vocabularies for observations and actions, the families of trace languages definable by regular, pushdown-style, linear-bounded, and Turing agent schemata form a strict hierarchy (each class strictly contains the previous for suitable trace encodings).*

Remark 5.2. *Proof sketches follow classical strict inclusions for regular $\subset$ context-free $\subset$ context-sensitive $\subset$ recursively enumerable languages, lifted to trace encodings; full proofs are standard and omitted for brevity.*

6 Evaluation and Pre-Registration

Following multi-model evaluation practice, we separate *claims* about an agent’s computational class from *evidence* (?).

6.1 Mapping implementations to schemata

- **Control-state audit:** bound $|S|$ (explicit modes in code: planner states, finite state machine nodes).
- **Memory audit:** classify $\mathcal{W}$ as finite, stack-structured, linear in transcript length, or unbounded external store.
- **Trace logging:** record (o_i, a_i) sequences under a fixed discretization of observations and actions; hash secrets for privacy.

6.2 Pre-registered hypotheses

Before comparing models, specify: (i) the trace encoding; (ii) which class $\mathcal{A}$ is claimed to inhabit; (iii) stopping rules and primary metrics (e.g., success rate within n steps for tasks known to separate classes). Changes to the audit protocol should be versioned in experiment notes, as with rubric versioning in structured evaluations.

6.3 Cost and latency

Report wall time and token usage per condition when comparing “regular” versus “pushdown-style” agent wrappers around the same backbone LLM, holding prompts and tool APIs fixed.

7 Discussion and Limitations

Metaphor versus identity. The Chomsky hierarchy classifies *string languages*. Agent traces are richer objects; our mapping is justified as a *structural analogy* on memory and control, not an identity theorem for natural-language generation by LLMs.

LLMs as transition noise. In practice, the neural policy may deviate from the specified schema; probabilistic extensions (stochastic δ) are left for future work.

Ethics. Formal power bounds do not imply safety; high-level Turing-complete wrappers around powerful tools require governance independent of this taxonomy.

8 Conclusion

We defined agent schemata whose trace languages parallel the Chomsky hierarchy: finite workspace (regular), stack-structured workspace (pushdown-style), linear-bounded workspace, and Turing-complete stores. This vocabulary supports precise comparisons of agentic programming patterns, pre-registered evaluations, and peer review that ask not only “which model?” but “under which memory and control assumptions?” Future work includes stochastic and resource-bounded refinements, and empirical case studies mapping deployed agents to these classes with public trace logs.

9 Research and Development Agent Model

9.1 Introduction

The *Research and Development (R&D) Agent Model* is a formal framework for autonomous agents that engage in iterative scientific research and development workflows. Inspired by the RD-Agent framework from Microsoft Research, this model introduces a systematic approach to agentic programs that can propose hypotheses, construct solutions, implement code, validate through testing, and improve through feedback.

The R&D agent cycle mirrors the scientific method:

1. **Hypothesis Generation** — Proposing ideas based on domain expertise
2. **Solution Construction** — Building models, features, or code from hypotheses
3. **Implementation** — Writing executable code
4. **Testing/Validation** — Running backtests, benchmarks, or experiments
5. **Feedback Analysis** — Evaluating results and identifying improvements
6. **Iteration** — Refining hypotheses based on feedback

This cycle is applicable across domains including quantitative finance, machine learning engineering, scientific research, and automated discovery.

9.2 Chomsky Hierarchy Formalization

Following the agent schema definition in Definition ??, an *Agent Schema* is:

$$\mathcal{A} = (S, s_0, \mathcal{W}_0, \Sigma_{\text{obs}}, \Gamma_{\text{act}}, \delta, F)$$

Where the components are defined as in Section ??.

The four-level hierarchy of agent schemata maps to R&D agents as follows:

Type	Automaton Class	Memory Constraint	Example R&D Agent
Type-3	Finite Automaton	No memory (Markovian)	Simple copilot agents
Type-2	Pushdown Automaton	Stack-based (nested calls)	Factor/Model evolution
Type-1	Linear-Bounded	O(n) workspace growth	Data science agents
Type-0	Turing Machine	Unbounded external store	Full R&D with knowledge base

9.3 Bayesian Loop Agent

We now introduce the *Bayesian Loop Agent*, which extends the standard R&D agent with explicit Bayesian inference at each cycle.

The loop follows: **Perceive** → **Train** → **Test** → **Eval** → **Observe** → **Learn** → **Repeat**

Definition 9.1 (Bayesian Agent Schema). *A Bayesian Loop Agent is a tuple:*

$$\mathcal{B} = (S, s_0, \mathcal{W}_0, \Sigma_{\text{obs}}, \Gamma_{\text{act}}, \delta_B, F, \mathcal{H}, \pi_0, \theta)$$

Where:

- $\mathcal{H}$: Hypothesis space (parameter space)
- π_0 : Prior distribution over $\mathcal{H}$
- θ : Likelihood function $\theta : \mathcal{H} \times \mathcal{O} \rightarrow [0, 1]$
- δ_B : Bayesian transition relation incorporating posterior updates

Definition 9.2 (Bayesian State Space). *The Bayesian Loop Agent operates across nine states:*

$$S = \{s_p, s_t, s_{te}, s_e, s_o, s_l, s_r, s_c, s_\epsilon\}$$

Where states represent: *Perceive, Train, Test, Eval, Observe, Learn, Repeat, Complete, Error.*

Definition 9.3 (Memory Stack). *The workspace maintains Bayesian state:*

$$\mathcal{W} = \{D_{\text{train}} : \text{training data}, D_{\text{test}} : \text{test data}, h : \text{parameters}, \pi : \text{posterior distribution}, \mathcal{L} : \text{log likelihood}, \mathbb{E} :$$

With growth: O(k) where k is the number of parameters.

Definition 9.4 (Trace Language). *Observations alphabet:*

$$\Sigma_{\text{obs}} = \{DATA, MODEL, METRICS, LIKELIHOOD, STOP\}$$

Actions alphabet:

$$\Gamma_{\text{act}} = \{perceive, train, test, eval, observe, learn, repeat\}$$

Production rules:

$$\begin{aligned} O_{\text{data}} &\rightarrow a_{\text{perceive}} O_{\text{model}} \\ O_{\text{model}} &\rightarrow a_{\text{train}} O_{\text{test}} \\ O_{\text{test}} &\rightarrow a_{\text{test}} O_{\text{eval}} \\ O_{\text{eval}} &\rightarrow a_{\text{eval}} O_{\text{metrics}} \\ O_{\text{metrics}} &\rightarrow a_{\text{observe}} O_{\text{likelihood}} \\ O_{\text{likelihood}} &\rightarrow a_{\text{learn}} (O_{\text{repeat}} | O_{\text{stop}}) \\ O_{\text{repeat}} &\rightarrow a_{\text{repeat}} O_{\text{data}} \end{aligned}$$

This is a *Type-1 (linear-bounded)* language with $O(n)$ workspace growth.

Definition 9.5 (Bayesian Update Rule). *The posterior update follows Bayes' theorem:*

$$\pi_{t+1}(h|D) \propto \pi_t(h) \cdot \mathcal{L}(D|h)$$

Implemented via:

1. **Prior:** $\pi_t(h)$ — Current hypothesis distribution
2. **Likelihood:** $P(D|h)$ — Probability of data given hypothesis
3. **Posterior:** $\pi_{t+1}(h)$ — Updated distribution
4. **Evidence:** $P(D) = \int \pi_t(h) \mathcal{L}(D|h) dh$

9.4 Convergence Criteria

The Bayesian agent terminates when:

1. **Maximum iterations:** $t \geq T_{\text{max}}$
2. **Convergence:** $|\pi_t - \pi_{t-1}| < \epsilon$
3. **Divergence:** Error state reached

9.5 Applications

The Bayesian Loop Agent applies to:

- **Quantitative Finance:** Perceive (market data) → Train (factor training) → Test (backtesting) → Eval (Sharpe ratio) → Observe (realized returns) → Learn (posterior update) → Repeat
- **Machine Learning:** Perceive (dataset) → Train (gradient descent) → Test (validation) → Eval (accuracy/F1) → Observe (prediction errors) → Learn (hyperparameter posterior) → Repeat
- **Scientific Discovery:** Perceive (experimental data) → Train (hypothesis instantiation) → Test (experimental validation) → Eval (error metrics) → Observe (measurements) → Learn (parameter posterior) → Repeat

9.6 Conclusion

The Bayesian Loop Agent provides a principled framework for R&D automation that explicitly accounts for uncertainty in hypothesis spaces. By maintaining a posterior distribution and updating via Bayesian inference, agents can quantify uncertainty, incorporate prior knowledge, adaptively explore the hypothesis space, and converge probabilistically rather than deterministically. This formalization extends the Chomsky hierarchy agent schema with probabilistic reasoning, yielding a powerful abstraction for autonomous scientific discovery.